Department-Aware Arbitration of Contended Enterprise Network Slices on a Live Traffic Testbed

Thrishwant G
[Department]
[Institution]
City, Country
email@institution.edu

K Jaya Sai Venkatesh
[Department]
[Institution]
City, Country
email@institution.edu

Hithesh Y
[Department]
[Institution]
City, Country
email@institution.edu

K J Sathya Prathmesh
[Department]
[Institution]
City, Country
email@institution.edu

October 21, 2026

Abstract

Enterprise networks provision more slice bandwidth than their links can carry. When demand peaks together, a shared bottleneck queue fills and every slice suffers, including the ones the business can least afford to lose; a generic controller then degrades slices by provisioning-time labels such as slice type or a static priority. We built a small but real testbed to study runtime arbitration under these conditions. Phones and laptops join one of seven department slices over a laptop's Wi-Fi hotspot; every byte they download passes through an application-layer gateway that enforces a per-slice token bucket in front of a shared 1 Mbit/s link and measures throughput, drops and probe round-trip latency every 2 s. A controller reads these measurements, detects contention, chooses new limits with Convex Business-Harm Arbitration (CBHA), a marginal-harm water-filling method driven by department business attributes and hard compliance ceilings, and pushes them back to the gateway, recording an auditable explanation for every decision. On a loopback run of six automated test cases, all checks pass. Under a lunch-hour overload of 1,316 kbit/s on a 950 kbit/s link, switching arbitration on cut the 95th-percentile latency of the clinic, factory and laboratory slices from about 505 ms to 22–23 ms and their loss from up to 21% to zero, while a signage slice labelled URLLC was held at its 28 kbit/s floor. We also show that weighted fair queueing without a controller protects light slices but is business-blind: at full peak it gives the life-safety clinic 37.5 kbit/s, below its 40 kbit/s contractual floor.

Keywords: network slicing, enterprise networks, testbed, token bucket, deficit round robin, resource arbitration, SLA, explainability

1 Introduction

Network slicing divides one physical network into logical networks, each with its own service-level agreement (SLA) on throughput, latency and loss [1–3]. Enterprise and private 5G sites use it to separate clinical telemetry, production control, laboratory instruments, video surveillance, office traffic and building services on one site network. Slices are given a guaranteed floor and a nominal (baseline) share, and the baselines deliberately add up to more than the link: statistical multiplexing makes this efficient because departments rarely peak together. When they do, the network is *contended*.

Two things then go wrong. First, without any arbitration the shared bottleneck queue fills and *every* slice waits behind it, so even a slice that sends far less than its share sees its latency rise to the length of the buffer [4]. Second, when a controller does intervene, it usually ranks slices by network attributes fixed at provisioning time: the slice/service type, the 5QI of its flows or a static priority integer [5, 6]. These labels describe traffic behaviour, not what the enterprise depends on. In the enterprise we use throughout this paper, canteen digital signage was provisioned as URLLC, the same type as patient telemetry, while a regulated quality laboratory was provisioned as eMBB; a label-based controller protects the signage and cuts the laboratory.

Our companion work formulated arbitration as minimisation of a convex *business-harm* function under hard compliance constraints and evaluated it in simulation.

Simulation leaves open whether the method works on real traffic: whether measured demand and latency are good enough inputs, whether the control loop is fast and stable, and whether cutting the right slices actually restores the protected ones. This paper answers those questions on a live testbed. Our contributions are:

- A low-cost slicing testbed in which real devices on a Wi-Fi hotspot generate traffic through an application-layer gateway with per-slice token buckets, a shared bottleneck and in-band latency probes, controlled through a small southbound API (Section 4).
- The integration of CBHA into a closed loop on measured data, including a shortage estimate based on carried rather than requested traffic (Section 3).
- Six reproducible test cases and an A/B experiment that quantifies what arbitration buys on the wire, with every 2 s measurement window recorded (Sections 5–6).
- A comparison with weighted fair queueing (deficit round robin) without a controller, showing where it is sufficient and where it breaks a business rule (Section 6.4).

2 Background and Related Work

Slicing and QoS. Surveys by Foukas *et al.* [2] and Afolabi *et al.* [3] cover slicing architectures and isolation. 3GPP defines slice identity in TS 23.501 [5] and per-session aggregate maximum bit rates in TS 23.503 [6]; the GSMA template describes slices by network attributes [7]. Differentiated Services [8] classifies traffic into a small number of behaviour classes. None of these carries the business context of the department that owns a slice.

Shaping and scheduling. Per-flow rate limits are enforced with token (leaky) buckets [9]. Fair queueing [10] and its efficient approximation, deficit round robin (DRR) [11], share a bottleneck among queues in proportion to weights and isolate a light flow from heavy ones. Large FIFO buffers at a bottleneck cause persistent queueing delay (bufferbloat) [4], which active queue management such as CoDel addresses [12]. These mechanisms decide how a link is shared *given* the limits; they do not decide which department should give up bandwidth.

Allocation and testbeds. Multi-tenant slice allocation has been studied as a game among tenants [13] and as forecasting-driven admission [14]; separable convex resource allocation is solved by greedy marginal allocation [15] and water-filling [16]. Emulated testbeds such as Mininet [17] and open 5G stacks [18,19] run real protocol code but need Linux hosts and radio emulation. Our testbed trades realism of the radio for the ability to use unmodified phones and a single Windows laptop.

3 Arbitration Model

We summarise CBHA as used here; the companion work gives the derivation. There are n slices on an offered capacity C. Slice i has baseline b_i, floor f_i, current limit a_i and measured demand D_i, and belongs to a department with seven business attributes: criticality, downtime cost per minute c_i, business importance, compliance level, customer impact, operational importance and recovery tolerance.

Business priority score. Each attribute is mapped to $[0, 100]$ and combined with policy weights (0.26, 0.20, 0.16, 0.14, 0.12, 0.07, 0.05 in the order above) into a score B_i; downtime cost is normalised logarithmically so no single department can saturate the score by claiming an extreme cost. Compliance also sets a hard ceiling M_i on how deep a slice may be cut: 0% for life-safety, at most 20% for regulated, otherwise the department's declared limit.

Harm and problem. Cutting slice i by a fraction d_i of its baseline causes harm

$$H_i(d_i) = B_i \, (d_i/M_i)^{\gamma_i} \, S_i \, F_i \, U_i \, I_i, \tag{1}$$

where the curvature $\gamma_i \in \{3.0, 2.2, 1.6, 1.15\}$ follows recovery tolerance (zero to high), S_i rises with live SLA stress, F_i is a bounded fairness term for slices that were cut recently, U_i converts expected hold time and downtime cost into urgency and I_i scales by recovery intolerance. CBHA solves

$$\min_{\mathbf{d}} \textstyle\sum_i H_i(d_i) \;\text{ s.t. }\; \sum_i r_i(d_i) \geq \Delta, \;\; 0 \leq d_i \leq \bar{d}_i, \tag{2}$$

with $\bar{d}_i = \min(M_i, (a_i - f_i)/b_i)$, by releasing bandwidth in small quanta to whichever slice has the lowest marginal harm per kbit/s freed. For $\gamma_i \geq 1$ the problem is convex and this greedy water-filling is optimal up to the quantum [15, 20]. Life-safety slices ($M_i = 0$) are removed before optimisation, so no weighting can reach them.

Shortage on measured traffic. On a real link a slice can never carry more than its limit, and the excess it requests is dropped at its own queue. Cutting the limit of a slice that already sends less than its new limit frees nothing. We therefore compute the shortage from carried traffic,

$$\Delta = \textstyle\sum_i \min(D_i, a_i) - (1 - h)\, C, \tag{3}$$

with headroom $h = 3\%$, and count as freed r_i only the part of a cut below $\min(D_i, a_i)$. A cut of a slice that cannot help is never chosen. When the ceilings do not allow Δ to be freed, CBHA frees what policy permits, reports a PARTIAL verdict with the shortfall and alerts the operator instead of breaking a rule.

Explanation. Every decision prices four alternative strategies on the same harm function (priority sort, uniform cut, single victim, protect-stressed), stores per-slice reasons that name concrete values, a confidence score, and the resolved policy version, and is written to an append-only audit log.

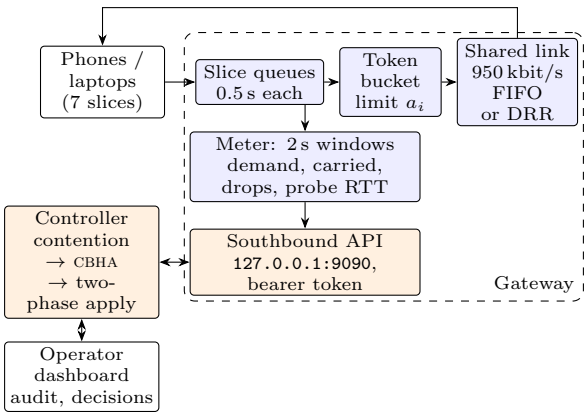


Figure 1: The testbed. Devices reach the gateway over HTTP on port 8080; every byte they download passes through the slice queue, its token bucket and the shared link, and returns with the probe replies (top arrow). The controller only reads measurement windows and writes limits through the southbound API.

4 Testbed Design

Fig. 1 shows the testbed. All components are TypeScript on Node.js and run on one Windows laptop; devices need only a web browser.

4.1 Gateway data path

A device joins a slice with a six-character join code (or a QR code shown on the dashboard) and opens a streaming download whose rate it chooses: that rate is the slice's *demand*. Every 20 ms the gateway's scheduler runs three steps. *Arrivals*: each session puts the bytes it requested into its slice queue in chunks of at most 1 KB; a full queue tail-drops. *Policing*: a per-slice token bucket releases bytes at no more than the slice's limit; its depth is $\max(0.1\,\text{s} \cdot a_i, 1\,\text{KB})$, so that a slice cut to a few kbit/s can still send a full chunk. *Transmit*: a root token bucket drains the link at the offered capacity. With the default FIFO discipline, policed chunks wait in one shared queue holding 0.5 s of link traffic, as in a plain switch port. With DRR, each slice's queue is served directly by weighted deficit round robin [11], with a quantum of 20 ms of traffic at the slice's limit. Each slice queue holds 0.5 s of traffic at the slice's *current* limit and is resized whenever the limit changes. Rates are computed from real elapsed time, so Windows timer jitter does not change them.

4.2 Measurement

Each device sends a small probe through the same slice queue as its data (every second from a phone, every 0.5 s from the test agents), so the probe's round-trip time is the latency the slice's traffic actually sees. The gateway closes a measurement window every 2 s on a timer and keeps, per slice: demand, carried and dropped rate, queueing delay, probe latency p50 and p95, jitter, loss and availability. Loss is the larger of the dropped share of offered bytes and the share of unanswered probes; an unknown value is reported as unknown, never as zero. `GET /v1/stats` returns the last completed window and is read-only, so the controller, the dashboard and the recorder all see the same numbers.

4.3 Control loop

The controller polls the gateway every 2 s. A contention opens after two consecutive windows in which demand exceeds 102% of offered capacity, and closes after three windows below it; a 20 s cooldown separates decisions. On a decision, slice updates, the decision record and its audit entry commit together, and the new limits are then pushed with `PUT /v1/slices/:code/limit`; a refusal is reconciled to the value the gateway reports. Degraded slices return to baseline when their department's recovery window (30 s to 30 min) expires or when an administrator overrides the decision with a stated reason. If the gateway restarts, the controller detects the lower uptime and re-sends the slice catalogue with the current limits.

4.4 Devices, operator and scale

Devices connect to the laptop's Windows Mobile Hotspot (at most eight clients) and open the department page served by the gateway on port 8080; the southbound port is bound to loopback. The page shows the slice's limit, demand, carried rate, latency and, when the slice is cut, the controller's explanation. An operator can set a slice's demand remotely, run a scenario that sets all slices at once, turn automatic arbitration off and on, remove a device and download the recording. The link is scaled to 1 Mbit/s so that seven phones on one hotspot can overload it without the Wi-Fi itself becoming the bottleneck; the engine's bandwidth unit is mapped to 1 kbit/s.

5 Experimental Setup

Table 1 lists the slices. Baselines total 1,480 kbit/s, 1.56 times the 950 kbit/s offered by the 1 Mbit/s link (50 kbit/s reserved); floors total 473 kbit/s, so the floors alone always fit. Note the label inversions: Canteen and Warehouse are URLLC but have low business scores, and the regulated Lab is eMBB.

Table 2 lists the test cases. An automated driver (`npm run demo:local`) starts the gateway and controller on one laptop, joins one software agent per slice over loopback, runs TC1–TC6 and checks every criterion from the recorded windows. In TC6 each phase has a 25 s warm-up followed by 40 s (20 windows) of measurement. Unless

Table 1: The enterprise: seven slices on a 950 kbit/s link

Slice (department)	Label	b_i	f_i	M_i	B_i	SLA p95
Patient telemetry (Clinic)	URLLC	60	40	0%	96.0	150 ms
Factory automation	URLLC	380	170	15%	81.7	180 ms
Lab instrumentation	eMBB	180	50	20%	70.9	500 ms
Warehouse operations	URLLC	160	55	35%	59.1	250 ms
CCTV (Security)	eMBB	240	60	40%	48.8	900 ms
Office Wi-Fi (Office IT)	eMBB	320	70	45%	43.4	600 ms
Canteen signage	URLLC	140	28	80%	8.0	350 ms
Total		1480	473			

Rates in kbit/s. M_i: maximum cut. B_i: business priority score.

Table 2: Test cases (demand as a fraction of baseline)

Case	Load	Pass criteria
TC1 Normal day	all 55% (814 kbit/s)	no contention, no cut, loss <1%
TC2 Canteen flood	Canteen 200%, rest 55%	Canteen held at its limit; no limit changes
TC3 Lunch rush	Canteen 200%, Office, CCTV 100%, Warehouse 80%, rest 55% (1,316 kbit/s)	decision enforced; Clinic not cut; floors and ceilings kept; protected slices within SLA
TC4 Peak	all 100% (1,480 kbit/s)	PARTIAL verdict and alert; Clinic not cut
TC5 Recovery	back to 55%	contention closes; recovery scheduled; override restores baselines
TC6 A/B	TC3 load, arbitration off then on	lower p95, no higher loss for Clinic, Factory, Lab

stated, the link uses FIFO. The agents use the same join, load and probe API as the phones; the hotspot run with phones is described in the repository's runbook and is not part of the measurements below.

6 Results

6.1 Shaper accuracy

We drove a single saturated slice (demand twice its limit) for 10 s of virtual time with uneven ticks of 20, 35 and 50 ms, imitating Windows timers. The carried rate was within 0.25% of the limit at every tested limit (Table 3), including the 28 kbit/s floor of the canteen slice.

6.2 Test cases TC1–TC5

All checks passed. In TC1 every slice carried its demand with p95 latency of 22–37 ms and no loss. In TC2 the canteen asked for 278 kbit/s and its own token bucket held it at 139 kbit/s; the canteen alone saw queueing (p95 465 ms, 50% loss), the other six slices kept p95 below 30 ms, and the engine made no change because the carried total (877 kbit/s) still fitted the link: the per-slice limit already isolates a single flooding department.

Table 3: Shaper accuracy on a saturated slice

Limit (kbit/s)	20	28	60	140	380	950
Error (%)	0.00	−0.21	0.10	0.25	−0.03	0.03

Table 4: TC6: lunch-rush load, arbitration off vs. on (FIFO link, means of 20 windows)

Slice	p95 latency (ms) off	p95 latency (ms) on	Loss (%) off	Loss (%) on	Limit on (kbit/s)
Clinic	503	**23**	5.62	**0**	60
Factory	509	**22**	21.47	**0**	380
Lab	505	**22**	14.99	**0**	180
Warehouse	507	475	18.29	5.23	122
CCTV	626	479	17.53	14.88	204
Office	573	476	19.81	31.19	221
Canteen	949	415	65.15	90.03	28

In TC3 demand rose to 1,316 kbit/s, and the carried total that the limits allow to 1,173 kbit/s, a shortage of 251.5 kbit/s against the 921.5 kbit/s target. A contention was already open from TC2 (demand above 102% of capacity, but nothing worth freeing), and new limits were enforced 4.0 s after the TC3 load started. CBHA cut Canteen to its 28 kbit/s floor (−80%), Office to 217 kbit/s, CCTV to 211 kbit/s and Warehouse to 121 kbit/s, and left Clinic, Factory and Lab untouched. The cut slices carried exactly their new limits; Clinic, Factory and Lab ran at 21–29 ms p95 with no loss. The URLLC label of the canteen did not protect it, and the eMBB label of the lab did not expose it.

In TC4 every slice demanded its full baseline. The ceilings allow at most 501 kbit/s to be freed against a 558.5 kbit/s shortage, so the engine reported PARTIAL, cut every slice except the clinic to its ceiling and alerted the operator. Because the link stays overloaded by about 30 kbit/s, every slice queued (p95 0.6–1.1 s), the honest outcome when policy forbids a feasible plan. The clinic, which demanded slightly more than its own 60 kbit/s limit, was held at that limit and was never cut. In TC5 the contention closed after three windows below the threshold, a recovery time was scheduled for every degraded slice, and an administrator override restored all baselines.

6.3 Arbitration off versus on (TC6)

Table 4 and Fig. 2 show the A/B experiment. With arbitration off, the carried total sits at the link rate (940–960 kbit/s) and the shared FIFO stays full. Every slice waits about one buffer length, 0.5 s, regardless of how little it sends; the clinic, which uses only 34 of its 60 kbit/s, loses 5.6% of its traffic and is more than three times over its 150 ms target. With arbitration on, the engine makes the same kind of decision as in TC3, carried traffic falls to about 920 kbit/s (the 3% headroom), and Clinic, Factory and Lab return to 22–23 ms p95 with zero loss, a

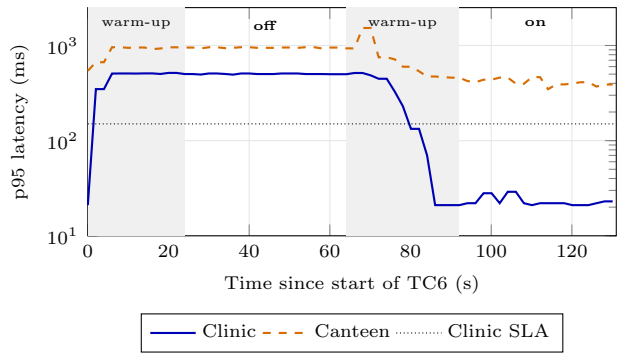


Figure 2: TC6 time series. With arbitration off the shared queue is full and the clinic waits about 0.5 s. After arbitration is switched on (64 s) the cheap slices are cut and the shared queue drains in about 20 s.

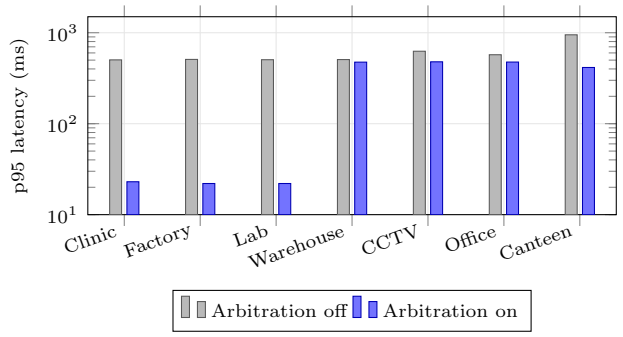


Figure 3: p95 latency per slice under the lunch-rush load (means of 20 windows).

22-fold reduction. The tail-drop loss is moved onto the slices the business chose to cut: Office and Canteen lose more than before, because their limits are now below their demand.

The slices that were cut still show about 475 ms p95. That is not shared-queue delay: each now waits in its *own* slice queue, which holds 0.5 s at its new limit and is permanently full because its demand exceeds its limit. The canteen's latency is lower than with arbitration off because it no longer waits behind the shared queue as well.

Recovery time. The protected slices took about 22 s to recover after arbitration was switched on (Fig. 2): roughly 4 s to detect and decide and 18 s to drain. The drain time follows from the design: the shared buffer holds $0.5\,\text{s} \times 950 = 475$ kbit and, after the cut, drains at only the headroom, $950 - 921 \approx 29$ kbit/s, so it needs about 16 s. A larger headroom or a smaller buffer would shorten recovery at the cost of utilisation; active queue management [12] at the shared link would remove most of the standing queue.

Table 5: No controller: carried rate (kbit/s) and probe p95 (ms) with every slice at its baseline limit

	TC3 lunch rush				TC4 everyone at peak			
	FIFO		DRR		FIFO		DRR	
Slice	kb/s	ms	kb/s	ms	kb/s	ms	kb/s	ms
Clinic	31.6	500	33.2	40	51.8	960	**37.5**	760
Factory	163.8	500	209.3	40	215.3	560	243.0	760
Lab	86.9	500	99.2	40	132.6	740	114.6	760
Warehouse	102.8	500	111.7	700	119.0	780	101.7	760
CCTV	207.6	660	167.9	700	137.6	660	153.0	760
Office	253.1	600	224.2	680	182.1	600	204.3	760
Canteen	97.6	960	97.6	700	105.8	820	88.8	760

6.4 Is fair queueing enough?

The FIFO result raises an obvious question: would weighted fair queueing at the link, with no controller at all, give the same protection? We ran the real gateway scheduler on a virtual clock with every slice at its baseline limit, for the TC3 and TC4 loads (Table 5).

Under the lunch rush, DRR alone protects Clinic, Factory and Lab: each sends less than its weighted share, so it never waits behind another slice's backlog (p95 40 ms, i.e. two scheduler ticks on the virtual clock). This is the isolation fair queueing is designed for [10, 11], and it is a strong argument for using it under any controller. But DRR shares by weight, not by business value. The canteen keeps 97.6 kbit/s, three and a half times its floor, while the warehouse, with a much higher business score, is squeezed below its demand. At full peak (TC4), DRR gives every slice its proportional share, $950 \cdot b_i/1480$: the life-safety clinic receives 37.5 kbit/s, below its 40 kbit/s contractual floor and in breach of its 0% ceiling, and every queue in the gateway is full. No choice of weights fixes this in general, because the right answer depends on which departments are active.

We repeated all six test cases on the testbed with the DRR link (`paper/results/drr-demo-local.csv`). TC1–TC5 passed. TC6 failed one check by design: with DRR the protected slices were already at 32–40 ms p95 with arbitration off, so switching it on could not lower their latency further. What arbitration changed was *who pays*: the canteen fell from 99 to its 28 kbit/s floor and the warehouse, CCTV and office loss dropped (13.0→5.3%, 30.0→16.5%), with the same engine decision as on FIFO. Fair queueing and business-aware arbitration are therefore complementary: DRR isolates slices within their share, CBHA decides the shares.

7 Discussion and Limitations

Loopback, not Wi-Fi. The measurements above use software agents on the same laptop as the gateway. They exercise the real gateway, measurement and control loop, but not the wireless hop. The 21–22 ms floor of the latency results is almost entirely the gateway's 20 ms

scheduler tick. The phone run on the hotspot is supported (QR join, per-tab identity, automatic rejoin after a gateway restart, firewall scripts, a session recorder) but its results are future work; on Wi-Fi we expect a higher and noisier latency floor and the same queueing behaviour at the gateway.

Application-layer slicing. The gateway shapes traffic it generates itself, at the application layer, in Node.js. It is not kernel traffic control, not 3GPP radio slicing, and its token buckets are not per-packet policers. An optional forward proxy admits joined devices' own traffic but does not shape it yet, and only downlink load is generated. The scaled 1 Mbit/s link keeps the Wi-Fi from being the bottleneck; results at higher rates would need a native shaper.

Demand is observable here. The gateway sees the rate each device requests before policing, which a real network does not: once a slice is capped it reports throughput at the cap. Our carried-traffic shortage partly addresses this, but a deployment would have to estimate demand from drops and queue signals.

Harm model. CBHA is optimal for its own harm function. The testbed shows that its decisions have the intended effect on the wire; it does not show that the harm function matches what each department would actually lose, which needs calibration against real outages.

8 Conclusion

A small testbed with real traffic, a shaped shared link and in-band probes is enough to show both halves of the contended-slice problem. Without arbitration, a full shared queue gives every department half a second of delay and real loss, the clinic included. With department-aware arbitration, the controller detects the overload within seconds, cuts the slices the business values least within their policy ceilings, restores the protected slices to their unloaded latency with zero loss, and records why. Fair queueing is a valuable complement, since it isolates slices that stay within their share, but it cannot decide who should give way at peak and can break a contractual floor. Next steps are the hotspot run with seven phones, a native kernel shaper at higher rates, active queue management at the shared link and demand estimation from capped traffic.

Code and Data Availability

The testbed, the test driver, the raw 2 s measurement windows behind every table and figure, and the scripts that produce them are available at [repository URL].

References

[1] NGMN Alliance, "5G white paper." NGMN Alliance, Feb. 2015.

[2] X. Foukas, G. Patounas, A. Elmokashfi, and M. K. Marina, "Network slicing in 5G: Survey and challenges," *IEEE Communications Magazine*, vol. 55, no. 5, pp. 94–100, 2017.

[3] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwarization: A survey on principles, enabling technologies, and solutions," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 2429–2453, 2018.

[4] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, 2012.

[5] 3GPP, "System architecture for the 5G system (5GS); stage 2," Technical Specification TS 23.501, 3rd Generation Partnership Project, 2022. Release 17.

[6] 3GPP, "Policy and charging control framework for the 5G system (5GS); stage 2," Technical Specification TS 23.503, 3rd Generation Partnership Project, 2022. Release 17.

[7] GSMA, "Generic network slice template." Permanent Reference Document NG.116, 2022.

[8] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, "An architecture for differentiated services." IETF RFC 2475, 1998.

[9] J. S. Turner, "New directions in communications (or which way to the information age?)," *IEEE Communications Magazine*, vol. 24, no. 10, pp. 8–15, 1986.

[10] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," in *Proc. ACM SIGCOMM*, pp. 1–12, 1989.

[11] M. Shreedhar and G. Varghese, "Efficient fair queuing using deficit round-robin," *IEEE/ACM Transactions on Networking*, vol. 4, no. 3, pp. 375–385, 1996.

[12] K. Nichols and V. Jacobson, "Controlling queue delay," *Communications of the ACM*, vol. 55, no. 7, pp. 42–50, 2012.

[13] P. Caballero, A. Banchs, G. de Veciana, and X. Costa-Pérez, "Network slicing games: Enabling customization in multi-tenant mobile networks," *IEEE/ACM Transactions on Networking*, vol. 27, no. 2, pp. 662–675, 2019.

[14] V. Sciancalepore, K. Samdanis, X. Costa-Pérez, D. Bega, M. Gramaglia, and A. Banchs, "Mobile traffic forecasting for maximizing 5G network slicing resource utilization," in *Proc. IEEE INFOCOM*, pp. 1–9, 2017.

[15] T. Ibaraki and N. Katoh, *Resource Allocation Problems: Algorithmic Approaches*. Cambridge, MA, USA: MIT Press, 1988.

[16] D. P. Palomar and J. R. Fonollosa, "Practical algorithms for a family of waterfilling solutions," *IEEE Transactions on Signal Processing*, vol. 53, no. 2, pp. 686–695, 2005.

[17] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010.

[18] Open5GS, "Open5GS: Open source implementation of 5G core and EPC." `https://open5gs.org`. Accessed Oct. 2026.

[19] A. Güngör, "UERANSIM: Open source 5G UE and RAN (gNodeB) simulator." `https://github.com/aligungr/UERANSIM`. Accessed Oct. 2026.

[20] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge University Press, 2004.